Generalización de Código de Automatización en Desarrollo de Software con IA

Tema 4. Integración de IA en IDE. Asistencia y mejoras en la escritura de código

Índice

Esquema

Ideas clave

4.1. Introducción y objetivos

4.2. Funcionalidades de asistencia en IDE

4.3. Uso de modelos de lenguaje en tiempo real

4.4. Sugerencias contextuales y autocompletado mejorado

4.5. Detección de errores y corrección automática

4.6. Personalización y configuración de asistentes en IDE

4.7. Referencias bibliográficas

A fondo

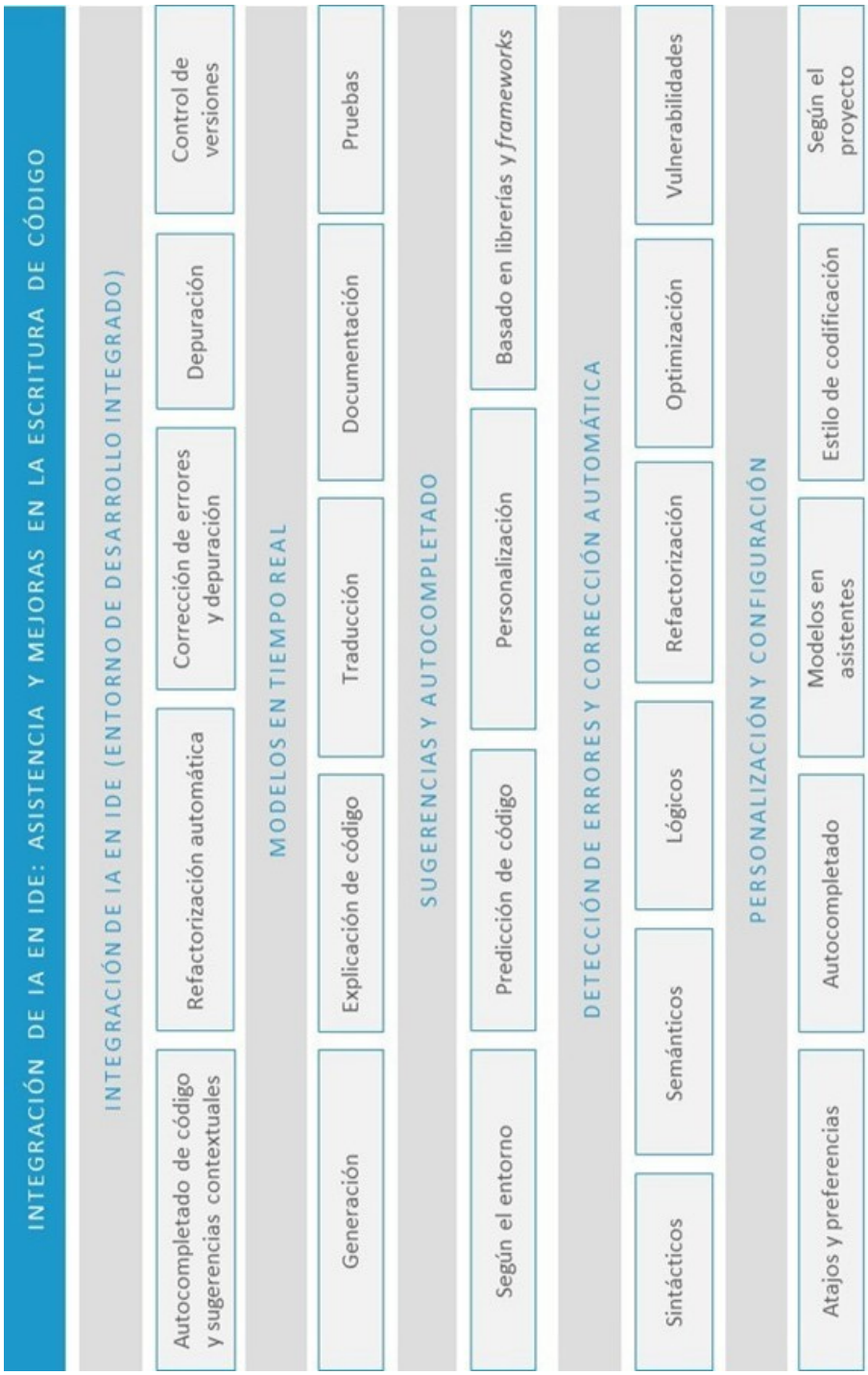
GitHub Copilot

JetBrains AI Assistant

Visual Studio y GitHub Copilot

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

4.1. Introducción y objetivos

La integración de inteligencia artificial (IA) en los entornos de desarrollo integrado (IDE) revolucionó la manera en que los programadores **escriben, depuran y optimizan el código.** Estas herramientas no solo les permiten a los desarrolladores **escribir código más rápido** y con menos errores, sino que también mejoran la calidad del código y la **eficiencia de los flujos de trabajo.**

Las **funcionalidades avanzadas** como el autocompletado inteligente, la detección automática de errores y la sugerencia contextual se convirtieron en herramientas indispensables para los programadores. Hay modelos de IA, como Codex, y herramientas, como GitHub Copilot, que permiten que los IDE **comprendan el contexto del código,** sugieran mejoras en tiempo real y **generen código** a partir de descripciones en lenguaje natural.

Esta nueva era de desarrollo impulsada por la IA no solo mejora la productividad, sino que también minimiza la carga cognitiva de los desarrolladores, de manera que les permite concentrarse en tareas más complejas, mientras que los IDE automatizan las tareas más repetitivas y rutinarias.

Los objetivos del tema son:

- Explorar cómo la IA mejora la asistencia en los IDE.
- Entender el impacto de los modelos de lenguaje en tiempo real.
- Implementar mejoras automatizadas y personalización en IDE.
- Evaluar la influencia de la IA en la calidad del código.

© Universidad Internacional de La Rioja (UNIR)

4.2. Funcionalidades de asistencia en IDE

Los IDE modernos evolucionaron para ofrecer una amplia **gama de funcionalidades de asistencia** que facilitan y optimizan el proceso de escritura de código. Estas son esenciales para **aumentar la productividad y la calidad del código.**

Desde el autocompletado hasta la depuración guiada, estas herramientas no solo ayudan a los desarrolladores a escribir código más rápido, sino que también facilitan l a **identificación y corrección de errores.** De esta manera, esto mejora significativamente el **proceso de desarrollo.** A continuación, se detallan las principales características de asistencia que ofrecen los IDE actuales.

Autocompletado de código

El autocompletado es una de las funcionalidades más fundamentales en los IDE. Ayuda a los desarrolladores al **sugerir posibles métodos, variables o clases** mientras escriben.

Esta funcionalidad no solo ahorra tiempo, sino que también reduce los errores tipográficos y de sintaxis.

© Universidad Internacional de La Rioja (UNIR)

Al escribir `system.out.` en IntelliJ IDEA para Java, el IDE sugiere métodos como `println()`, lo que permite seleccionar la opción correcta sin necesidad de recordar exactamente la sintaxis.

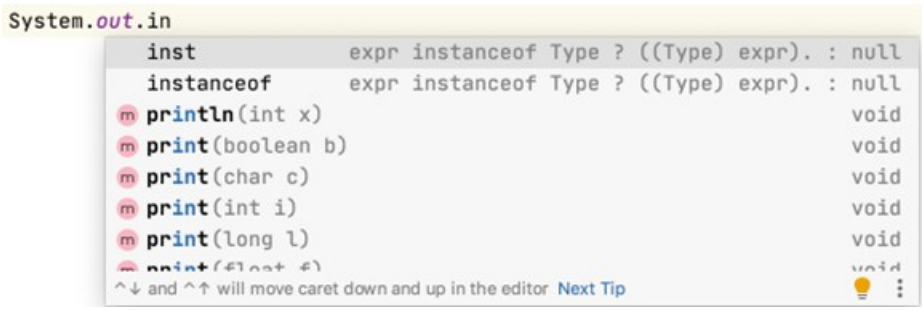


Figura 1. Sugerencias de autocompletado en IntelliJ. Fuente: JetBrains, 2025.

Sugerencias contextuales

Los IDE modernos utilizan inteligencia contextual para **sugerir fragmentos de código o métodos relevantes** basados en el código que el desarrollador está escribiendo en tiempo real. Estas sugerencias pueden ser especialmente útiles cuando el IDE detecta **patrones repetitivos** o estructuras comunes en el código. Algunos IDE, como PyCharm y Visual Studio Code (VS Code), utilizan **modelos de IA para predecir** mejor las necesidades del desarrollador.

Por ejemplo, en VS Code, si el programador crea un ciclo `for` para recorrer un *array,* el IDE puede sugerir la estructura completa del ciclo basada en el tipo de la variable.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Depuración guiada

Los IDE proporcionan herramientas avanzadas de depuración que les permiten a los desarrolladores ejecutar el código paso a paso, establecer puntos de ruptura *(breakpoints)*, observar el estado de las variables y ver cómo cambia el flujo del programa en tiempo real.

Las características destacadas son:

- Visualización en tiempo real del estado de las variables.
- Posibilidad de modificar valores de variables durante la ejecución.
- Visualización de la pila de llamadas para rastrear el flujo del programa.

Por ejemplo, en Visual Studio, el desarrollador puede pausar la ejecución en un punto específico y observar cómo las variables cambian con cada iteración de un ciclo.

Refactorización automática

La refactorización de código les permite a los desarrolladores **realizar modificaciones estructurales** en su código sin cambiar su comportamiento. Los IDE ofrecen herramientas que permiten **automatizar estas transformaciones,** lo que reduce el riesgo de errores al realizar cambios manuales.

© Universidad Internacional de La Rioja (UNIR)

Las funciones comunes son:

- Renombrar variables o métodos en todo el proyecto.
- Extraer métodos o clases.
- Optimizar *imports* y eliminar código redundante.

Por ejemplo, en Eclipse, el desarrollador puede seleccionar una variable y usar la opción de «Renombrar» para cambiarla en todas las instancias de su proyecto, para asegurar consistencia en el código.

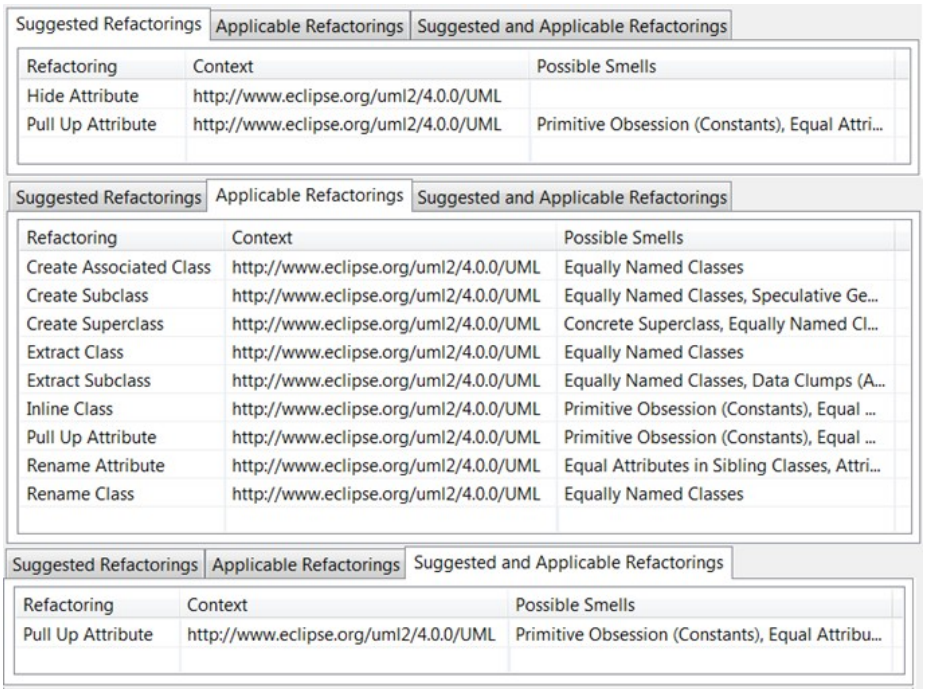


Figura 2. Sugerencias de refactorización en Eclipse. Fuente: Eclipse, s. f.

© Universidad Internacional de La Rioja (UNIR)

Corrección automática de errores

Los IDE avanzados proporcionan **herramientas de corrección automática,** que detectan errores de sintaxis o lógica en tiempo real y sugieren correcciones instantáneas. Estas herramientas son útiles para **corregir errores comunes,** como la falta de un punto y coma o el uso incorrecto de un tipo de variable.

Hay herramientas, como ReSharper para Visual Studio, que pueden incluso ofrecer mejoras de rendimiento del código.

Por ejemplo, si un desarrollador olvida importar una clase en PyCharm, el IDE subraya el error y sugiere automáticamente el *import* adecuado.

Integración con sistemas de control de versiones

Muchos IDE ofrecen **integración nativa** con sistemas de control de versiones como Git, lo que les permite a los desarrolladores **gestionar fácilmente el código fuente** sin salir del entorno de desarrollo. Esto incluye la posibilidad de realizar `commits`, **fusiones y revisiones de código** directamente desde el IDE.

Las características de este sistema son:

- Visualización de cambios en tiempo real.
- Posibilidad de realizar operaciones de Git (`commit` , `push` y `pull`).
- Comparación de versiones y fusión automática de conflictos.

Por ejemplo, en IntelliJ IDEA, los desarrolladores pueden realizar un `commit` y ver un historial visual de los cambios en el proyecto desde el panel de control de Git.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Asistentes de código basados en IA

En los últimos años, las herramientas basadas en IA, como GitHub Copilot, revolucionaron la forma en que los IDE **ayudan a los desarrolladores.** Estas herramientas son capaces de **generar bloques completos de código,** ofrecer sugerencias inteligentes basadas en el contexto del proyecto y **aprender del estilo de codificación del desarrollador.**

Por ejemplo, GitHub Copilot puede sugerir código para una función completa en Python solo con una descripción en lenguaje natural en el comentario.

Documentación y navegación rápida

Los IDE también permiten acceder fácilmente a la **documentación de métodos, de clases y de bibliotecas,** lo que ayuda a los desarrolladores a entender rápidamente el **propósito y uso de las funciones.**

Además, los IDE ofrecen herramientas de navegación rápida para moverse entre diferentes partes del código o buscar definiciones y usos de variables y funciones.

Por ejemplo, en Eclipse, al colocar el cursor sobre una función de una biblioteca, el IDE muestra la documentación relacionada y la firma de la función.

© Universidad Internacional de La Rioja (UNIR)

Plantillas y snippets de código

Otra funcionalidad común es la inclusión de **plantillas y *snippets* de código reutilizables** que permiten insertar rápidamente estructuras de código complejas. Estas plantillas pueden ser **configuradas por el propio desarrollador** o por la comunidad.

Por ejemplo, en VS Code, se puede escribir un fragmento como `for` y el IDE expande automáticamente el bucle `for` con la sintaxis completa, lo que ahorra tiempo de escritura.

© Universidad Internacional de La Rioja (UNIR)

4.3. Uso de modelos de lenguaje en tiempo real

El uso de modelos de lenguaje en tiempo real en IDE revolucionó la manera en que los desarrolladores **crean, depuran y optimizan código.** Estos modelos, impulsados por tecnologías de IA como los *transformers* (por ejemplo, GPT o Codex), ofrecen **sugerencias y correcciones automáticas,** mientras que el desarrollador trabaja.

Estas herramientas no solo mejoran la eficiencia al reducir el tiempo de escritura y depuración, sino que, al automatizar aspectos rutinarios del proceso, también les permiten a los desarrolladores centrarse en tareas más complejas.

A medida que estas tecnologías continúan evolucionando, se espera que su integración en los flujos de trabajo sea cada vez más **profunda y efectiva.** A continuación, se explican en detalle las **aplicaciones** de estos modelos en tiempo real, sus **beneficios** y su **impacto** en el flujo de trabajo de desarrollo.

Asistencia contextual y explicación de código

Los modelos de lenguaje en tiempo real también pueden proporcionar **explicaciones detalladas** de fragmentos de código existentes. Esto es especialmente útil cuando un desarrollador trabaja con **código heredado o bibliotecas desconocidas.** El modelo analiza el código y puede proporcionar **comentarios o explicaciones en lenguaje natural.**

Por ejemplo, si un desarrollador está trabajando con un código complejo en VS Code y no entiende cómo funciona una parte, puede pedirle al modelo que explique la lógica detrás de una función o bloque. El modelo responde con la generación de una explicación clara del propósito y comportamiento del código.

© Universidad Internacional de La Rioja (UNIR)

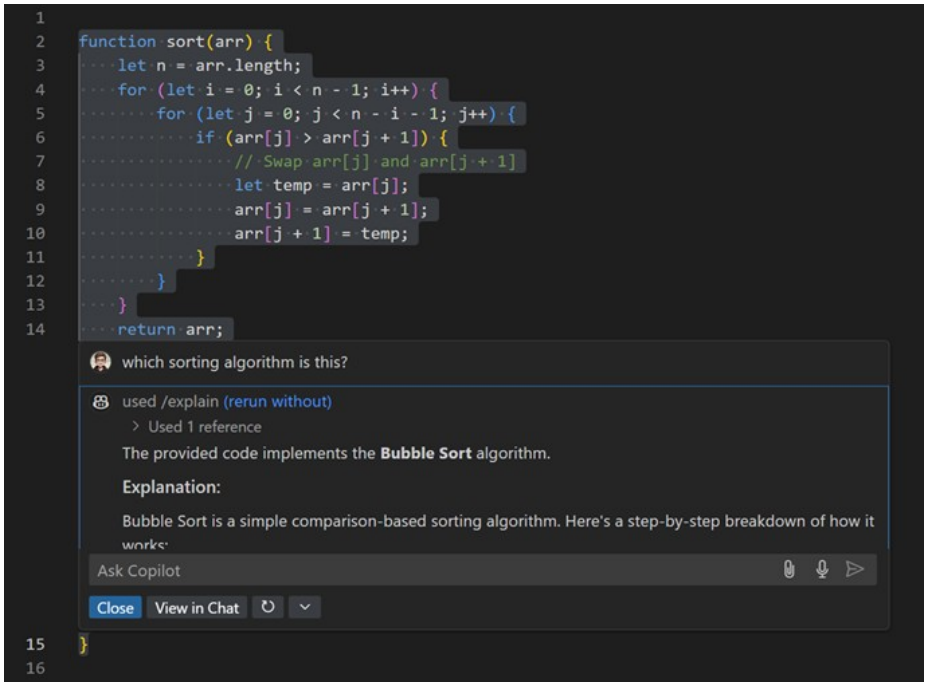


Figura 3. Ejemplo de explicación de código en VSCode con GitHub Copilot. Fuente: Visual Studio, 2025.

Generación de código a partir de lenguaje natural

Una aplicación destacada de los modelos de lenguaje en tiempo real es la capacidad de generar código directamente desde **descripciones en lenguaje natural.**

Esto significa que los desarrolladores pueden escribir comentarios o instrucciones en su idioma y el modelo transforma esas descripciones en código funcional.

Por ejemplo, en un IDE que utiliza Codex, si el desarrollador escribe en un comentario «crea una función para calcular el promedio de una lista de números», el modelo puede generar automáticamente el código en el lenguaje de programación correspondiente (como Python o Java).

© Universidad Internacional de La Rioja (UNIR)

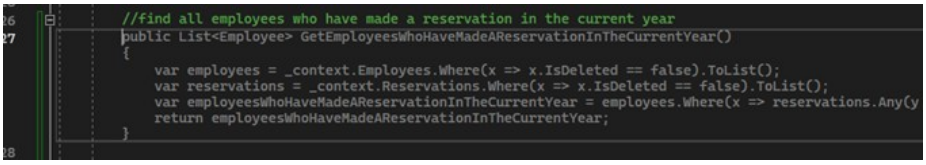


Figura 4. GitHub Copilot es capaz de sugerir código a partir de un comentario. Fuente: it-Labs, s. f.

Traducción entre lenguajes de programación en tiempo real

Una funcionalidad avanzada que los modelos de lenguaje ofrecen es la **capacidad de traducir código** de un lenguaje a otro en tiempo real. Esto es especialmente útil en entornos donde se necesitan **versiones de un mismo programa** en diferentes lenguajes o plataformas.

Por ejemplo, un desarrollador puede tener un algoritmo escrito en Python y necesita implementarlo en JavaScript. Al seleccionar el código en su IDE, el modelo traduce automáticamente el código a JavaScript, de manera que preserva la funcionalidad original.

Ayuda en la documentación del código

L a **generación automática de documentación** es otra función importante que proporcionan los modelos de lenguaje en tiempo real. Los desarrolladores pueden utilizar estas herramientas para **generar comentarios o documentación técnica** de forma instantánea, para asegurarse de que el código esté bien documentado y sea fácil de mantener.

Esto ahorra tiempo y garantiza que el código esté documentado de manera clara y coherente, una práctica crucial en proyectos a gran escala o de código abierto.

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo, en PyCharm, un desarrollador puede seleccionar una función sin comentarios y el modelo sugerirá una descripción detallada del propósito de la función, los parámetros utilizados y los posibles valores de retorno. De esta manera, genera documentación en formato estándar como docstrings .

Pruebas automáticas en tiempo real

Los **modelos de lenguaje** también pueden asistir en la generación automática de pruebas unitarias, mientras que se escribe el código. Esta funcionalidad garantiza que cada bloque de código tenga una **cobertura de pruebas adecuada,** lo que mejora la calidad del *software* y **facilita la detección de errores** futuros.

Por ejemplo, en VS Code, el modelo puede sugerir pruebas unitarias automáticamente basadas en una nueva función, para crear casos de prueba relevantes para diferentes escenarios posibles.

© Universidad Internacional de La Rioja (UNIR)

4.4. Sugerencias contextuales y autocompletado mejorado

Los IDE modernos avanzaron significativamente en términos de **asistencia inteligente** para los desarrolladores. Entre estas funcionalidades, se destacan las **sugerencias contextuales** y el **autocompletado mejorado** que permiten mejorar la productividad y la precisión en la programación.

El uso de sugerencias contextuales y autocompletado mejorado en los IDE no solo acelera el desarrollo de *software,* sino que también garantiza que el código sea más coherente, eficiente y libre de errores.

Estas herramientas, especialmente cuando se combinan con IA, son un **apoyo indispensable** para los desarrolladores modernos, ya que mejoran tanto la **calidad del código** como la **experiencia de desarrollo** en general. A continuación, se detallan estas características, cómo funcionan y su impacto en el flujo de trabajo de desarrollo.

Sugerencias contextuales: asistencia basada en el entorno del código

Las sugerencias contextuales son **recomendaciones de código** que se generan con base en el análisis del contexto en el que está trabajando el desarrollador. A diferencia del autocompletado tradicional, que simplemente sugiere posibles variables o métodos basados en coincidencias simples, las sugerencias contextuales utilizan **información más profunda** sobre el código en su conjunto, por lo que incluye **dependencias, estructuras y patrones anteriores.**

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo, si el desarrollador crea una función que manipula una lista en Python, las sugerencias contextuales en IDE, como PyCharm, pueden ofrecer métodos específicos de listas, como `append()` o `sort()`, basados en las interacciones previas que el código tuvo con objetos de ese tipo.

Esta funcionalidad reduce la carga cognitiva del desarrollador, ya que no tiene que recordar o buscar la sintaxis exacta de funciones que ya utilizó previamente. Además, mejora la coherencia del código al mantener patrones de uso consistentes.

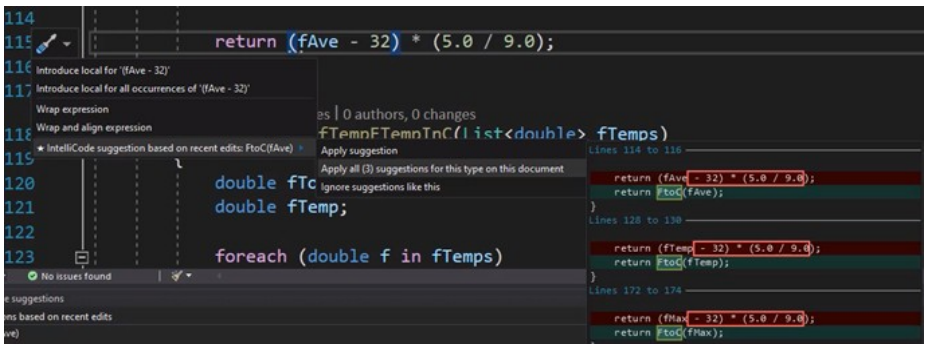


Figura 5. Sugerencias múltiples basadas en cambios anteriores del código con IntelliCode. Fuente: Groenewegen, 2020.

Personalización de las sugerencias

Algunos IDE permiten **personalizar** las sugerencias contextuales y el autocompletado según las preferencias del usuario o el proyecto en cuestión. Esto puede incluir configuraciones que **prioricen ciertos patrones** de uso o que incluyan **librerías específicas** que sean comunes en el proyecto actual.

Por ejemplo, en PyCharm, un desarrollador que habitualmente trabaja con Pandas puede configurar el autocompletado para priorizar funciones este programa cuando trabaje con estructuras como *dataframes,* lo que agiliza el proceso de análisis de datos.

© Universidad Internacional de La Rioja (UNIR)

En proyectos con varias dependencias o en entornos de trabajo colaborativo, estas sugerencias pueden **reducir el tiempo necesario** para buscar en la documentación o **aprender nuevas bibliotecas.** De esta manera, ofrecen una experiencia de programación más fluida y eficiente.

Mejoras en la precisión de las sugerencias

A medida que los modelos de lenguaje y los algoritmos de IA evolucionaron, la **precisión de las sugerencias de código** mejoró significativamente. Esto no solo reduce los errores de codificación, sino que también **fomenta la adopción de mejores prácticas.**

Por ejemplo, en JetBrains Rider, si un desarrollador utiliza una variable que puede devolver un valor nulo, el IDE puede sugerir automáticamente una verificación `null` para evitar excepciones inesperadas, lo que aumenta la robustez del código.

Estos sistemas también pueden **aprender del código** que se ha escrito en el proyecto, lo que hace que las sugerencias sean cada vez más **precisas y alineadas** con el estilo del equipo o del proyecto.

Sugerencias basadas en librerías y frameworks específicos

El autocompletado mejorado y las sugerencias contextuales también se integran con **librerías y *frameworks* específicos,** de manera que se adapta al entorno de trabajo. Esto es particularmente útil cuando se trabaja en proyectos que dependen de **marcos o API populares,** ya que los IDE pueden **sugerir configuraciones** o usos comunes de estos.

Por ejemplo, en VS Code, si un desarrollador trabaja con React, el autocompletado puede sugerir componentes, `hooks` y métodos específicos de este sistema en el contexto de una aplicación web, lo que reduce la necesidad de buscar en la documentación.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Al conocer las librerías que se utilizan, las sugerencias contextuales proporcionan fragmentos de código preconstruidos que son comunes en esas tecnologías, lo que acelera el desarrollo de nuevas funcionalidades.

Autocompletado mejorado: más allá de la sintaxis

E l **autocompletado mejorado** es una evolución del autocompletado básico que ofrecen los IDE. En lugar de solo sugerir variables o métodos con base en coincidencias parciales de nombre, este sistema se basa en **análisis semántico avanzado,** de manera que **predice** lo que el desarrollador necesita escribir a continuación en función del código anterior.

Por ejemplo, al escribir una función en VS Code o IntelliJ IDEA, los modelos de autocompletado mejorado, como los basados en IA, pueden generar automáticamente bloques de código completos, como bucles, condicionales o incluso casos de prueba, por lo que se basa en el comportamiento esperado del programa.

Hay herramientas, como GitHub Copilot, impulsadas por Codex (un modelo de lenguaje basado en GPT-3) que pueden **anticipar la intención del desarrollador,** por lo que generan **bloques de código complejos** con una precisión sorprendente.

© Universidad Internacional de La Rioja (UNIR)

4.5. Detección de errores y corrección automática

La **detección de errores y corrección automática** es una funcionalidad clave que mejoró drásticamente el flujo de trabajo en el desarrollo de *software.* Gracias al uso de la IA y las técnicas avanzadas de análisis estático y dinámico, los IDE modernos pueden **detectar errores de sintaxis, semántica y lógica** de forma automática, además de **proporcionar sugerencias** para su corrección en tiempo real.

Esta característica transformó el desarrollo de *software,* dado que hizo que el proceso sea más **eficiente, preciso y seguro.** Gracias a las tecnologías de análisis estático y dinámico, y de *machine learning,* los desarrolladores ahora pueden **confiar en sus herramientas** no solo para encontrar problemas, sino también para **aprender de los errores y mejorar** su código con mejores prácticas.

A medida que estos sistemas continúan evolucionando, se espera que la detección de errores sea aún más precisa, para **minimizar el esfuerzo humano** necesario para corregirlos. A continuación, se detallan los **tipos de errores** que pueden detectarse, cómo se corrigen y cómo se implementa esta funcionalidad en entornos de desarrollo.

Detección de errores sintácticos

Los errores de sintaxis son uno de los problemas más comunes que enfrentan los desarrolladores, especialmente en lenguajes estrictos en cuanto a estructura. Los IDE están equipados con analizadores sintácticos que detectan automáticamente estos errores, mientras que se escribe el código.

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo, si un desarrollador omite un punto y coma (;) en un lenguaje como Java o C++, el IDE resaltará inmediatamente la línea con el error, por lo que proporcionará una sugerencia para corregirlo.



Figura 6. IntelliJ puede detectar errores automáticamente y sugerir las posibles correcciones al mismo.

Fuente: IntelliJ IDEA, 2024.

Los IDE no solo notifican al desarrollador del error, sino que también pueden **ofrecer una corrección** con un solo clic. Por ejemplo, en Eclipse o IntelliJ IDEA, si falta un punto y coma al final de una línea, el IDE **sugerirá y añadirá el carácter** faltante automáticamente.

Detección de errores semánticos

Además de los errores sintácticos, los **errores semánticos** ocurren cuando el código es estructuralmente correcto, pero contiene operacio**nes que no tienen sentido lógico** o que son inconsistentes con el tipo de dato utilizado. Los IDE modernos, con la ayuda de modelos de IA y del análisis de tipo estático, pueden identificar estos errores y **sugerir correcciones** basadas en el contexto.

Por ejemplo, en un programa de Python, si un desarrollador intenta sumar una cadena de texto a un número, el IDE (como PyCharm) señalará este error semántico y sugerirá una conversión de tipo, como convertir el número a una cadena (`str(numero)`).

© Universidad Internacional de La Rioja (UNIR)

El IDE ofrece una solución directa, como envolver el número con una función de conversión. De esta manera, con un solo clic, el error se corrige y el programa se vuelve coherente.

Errores lógicos y sugerencias de refactorización

Los errores lógicos son más **complejos de detectar,** ya que el código puede funcionar técnicamente, pero no produce los resultados esperados. Algunos IDE, gracias a las **técnicas de análisis dinámico** y los patrones de código aprendidos a través de IA, pueden identificar estos problemas y **sugerir mejoras o cambios** en la lógica del programa.

Por ejemplo, en Visual Studio, si un bucle tiene una condición que no se cumple correctamente o si una operación dentro del bucle nunca se ejecuta, el IDE puede resaltar este patrón de código ineficiente y sugerir una refactorización de la lógica.

Algunos modelos de IA, como GitHub Copilot, no solo **detectan estos errores,** sino que también pueden **predecir el comportamiento correcto** basado en el código circundante o en los comentarios del programador.

Detección de errores de tipos y null safety

El **manejo de tipos** y de **valores nulos** es una fuente común de errores en muchos lenguajes de programación. Los IDE modernos, como JetBrains Rider o IntelliJ IDEA, utilizan el **análisis estático** para detectar problemas potenciales con *null pointers* o errores de tipo, y ofrecen soluciones proactivas.

Por ejemplo, en Kotlin, si un lenguaje que enfatiza la seguridad ante valores nulos *(null safety)*, el IDE resaltará cualquier variable que pueda contener un valor nulo sin ser gestionado adecuadamente. Además, sugerirá la adición de una verificación `null` o la conversión segura mediante `?.` o `!!`.

© Universidad Internacional de La Rioja (UNIR)

El IDE no solo detecta el problema, sino que **sugiere y aplica correcciones automáticas,** lo que asegura que el código maneje correctamente las posibles excepciones de valor nulo.

Sugerencias de optimización y mejora de código

Además de corregir errores, los IDE pueden **sugerir mejoras para optimizar el código,** desde simplificar expresiones hasta eliminar código redundante. Los **motores de análisis de código** revisan continuamente el código fuente en busca de patrones ineficientes y **ofrecen refactorizaciones** para mejorar el rendimiento o la claridad.

Por ejemplo, si un desarrollador usa una concatenación ineficiente de cadenas en un bucle en Java, el IDE (como Eclipse o IntelliJ IDEA) sugerirá el uso de `StringBuilder` en lugar de concatenar directamente con el operador `+`, lo que mejora significativamente el rendimiento.

Este tipo de sugerencias no solo corrigen errores, sino que también fomentan mejores prácticas de programación al enseñar a los desarrolladores a escribir código más eficiente y mantenible.

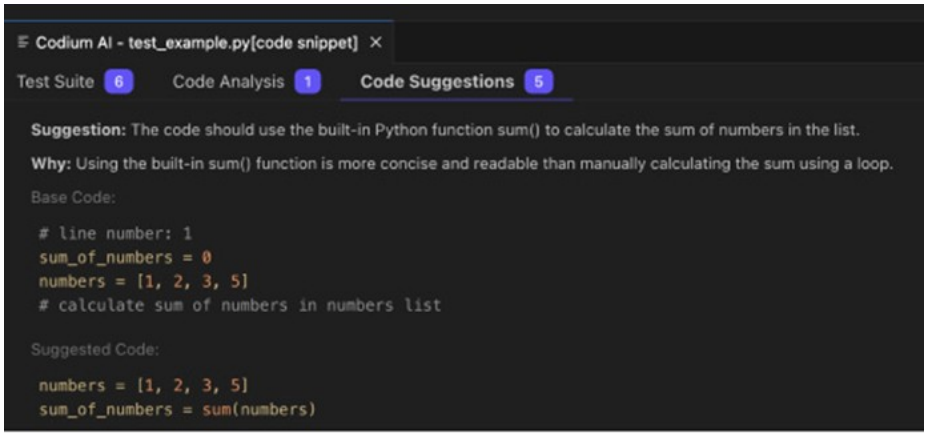


Figura 7. Sugerencia de optimización con CodiumAI. Fuente: Kenvel, 2023.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Detección de vulnerabilidades de seguridad

Los IDE integran cada vez más herramientas para la **detección de vulnerabilidades de seguridad** en tiempo real. Utilizan el análisis estático y los modelos de *machine learning* para identificar problemas como **inyecciones SQL** *(structured query language)*, **XSS** *(cross-site scripting)* o **fugas de datos sensibles** en el código.

Por ejemplo, si un desarrollador construye una consulta SQL directamente a partir de la entrada del usuario sin una validación adecuada, un IDE (como VS Code) con extensiones de seguridad alertará sobre el riesgo de una posible inyección SQL, de manera que sugerirá el uso de consultas preparadas o validación de entradas.

El IDE puede proporcionar **plantillas seguras** o ejemplos de código que mitiguen los riesgos, como el uso de sanitización o escapes apropiados para los datos de entrada.

Retroalimentación inmediata y experiencia de usuario

La clave de la detección de errores y corrección automática es la **retroalimentación inmediata** que los IDE proporcionan al desarrollador. En lugar de esperar a compilar o ejecutar el programa, los errores son detectados y corregidos, mientras que se escribe el código, lo que aumenta la productividad y **minimiza errores acumulados** a lo largo del desarrollo.

Algunos estudios mostraron que la detección temprana de errores reduce en un 50 % el tiempo total dedicado a la depuración en proyectos a gran escala.

© Universidad Internacional de La Rioja (UNIR)

Colaboración con herramientas de integración continua (CI) y despliegue continuo (CD)

Los IDE modernos también se integran con herramientas de CI y CD para detectar errores en tiempo real antes de que el código sea enviado al repositorio. Esto permite corregir errores antes de integrarse al *pipeline* de desarrollo, lo que garantiza que solo el código de alta calidad llegue al producto final.

Por ejemplo, en GitLab o Jenkins, las extensiones del IDE pueden ejecutar automáticamente tests y análisis de código al hacer commits o pull requests, para verificar errores y alertar a los desarrolladores de posibles problemas.

© Universidad Internacional de La Rioja (UNIR)

4.6. Personalización y configuración de asistentes en IDE

Los asistentes inteligentes en IDE transformaron la experiencia de los programadores, ya que ofrecen **sugerencias en tiempo real,** autocompletado y análisis de errores. Sin embargo, uno de los mayores beneficios de estos asistentes es la **capacidad de personalización y configuración,** lo que les permite a los desarrolladores **adaptar estas herramientas** a sus necesidades específicas y estilos de programación.

La personalización y configuración de asistentes en los IDE no solo mejora la productividad individual, sino que también permite adaptar la experiencia de programación a las necesidades específicas de los desarrolladores o los proyectos.

Con una correcta configuración, estos asistentes pueden convertirse en **herramientas poderosas** que maximizan la eficiencia y minimizan errores, al tiempo que se **adaptan al estilo y a las preferencias** de cada programador.

A continuación, se detalla cómo los asistentes en los IDE se pueden **personalizar y configurar** para mejorar la productividad, aumentar la precisión del código y mejorar la experiencia de desarrollo en general.

Configuración de atajos y preferencias de uso

Los IDE ofrecen asistentes que se pueden ajustar a través de **atajos personalizados** o configuración de teclas rápidas. Esto facilita la **interacción con las sugerencias,** la corrección de errores y otras funcionalidades, sin necesidad de abandonar el **flujo de trabajo.**

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo, en VS Code, es posible personalizar las combinaciones de teclas para acciones comunes, como activar o desactivar sugerencias automáticas, aceptar sugerencias de autocompletado o acceder rápidamente a plantillas de código. Esto permite que los programadores adapten el IDE a su flujo de trabajo y optimicen la velocidad de escritura.

Personalización del autocompletado de código

El **autocompletado** es una de las funcionalidades más usadas en los asistentes de código. Los desarrolladores pueden **configurar** qué tipos de sugerencias desean recibir, para priorizar aquellas que sean más relevantes para su contexto o **estilo de programación.**

Por ejemplo, en JetBrains IntelliJ IDEA, se puede configurar el asistente de autocompletado para que sugiera únicamente fragmentos de código que sean compatibles con el lenguaje y las bibliotecas que se están utilizando. También se puede ajustar para que ignore ciertas sugerencias o modifique el formato de estas.

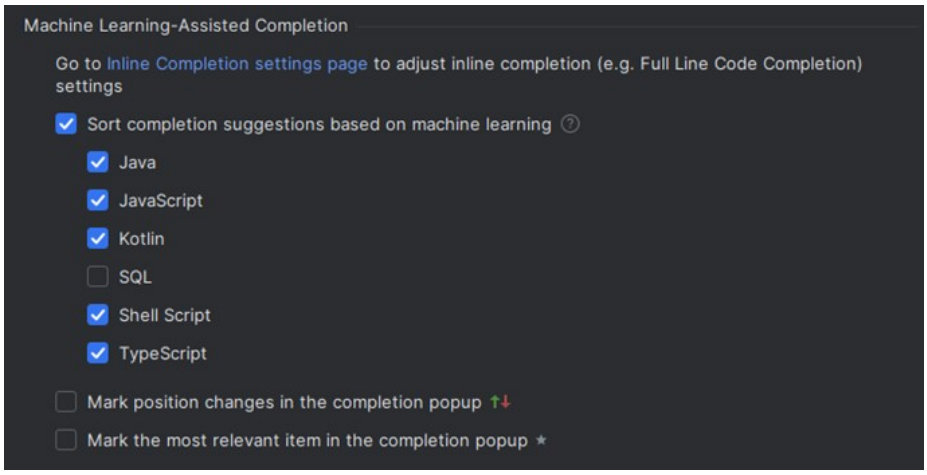


Figura 8. Opciones de configuración en IntelliJ. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Los programadores pueden ajustar el asistente para que sea más o menos intrusivo.

Por ejemplo, en PyCharm (otro IDE de JetBrains), se puede deshabilitar el autocompletado automático para que solo funcione bajo demanda.

Configuración de modelos de lenguaje en asistentes

Muchos asistentes están impulsados por modelos de lenguaje, como GPT-3, Codex o motores de IA específicos para programación. La capacidad de elegir o ajustar el modelo de lenguaje subyacente permite una mejor adaptación a las necesidades específicas del proyecto.

Por ejemplo, en GitHub Copilot, los usuarios pueden configurar la frecuencia y el tipo de sugerencias ofrecidas por el modelo, así como desactivar ciertos tipos de recomendaciones. Los desarrolladores también pueden configurar el asistente para optimizar su rendimiento en función de lenguajes específicos de programación.

© Universidad Internacional de La Rioja (UNIR)

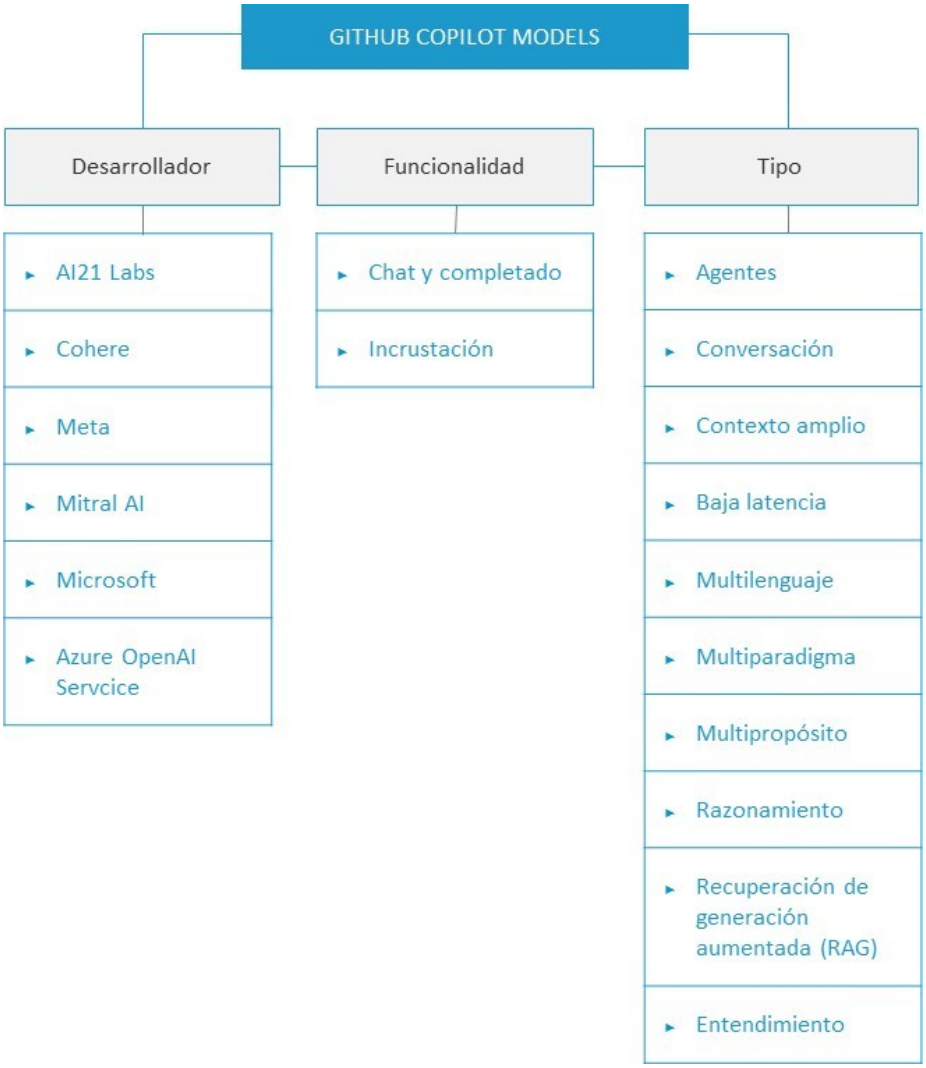


Figura 9. GitHub Copilot permite la integración con múltiples modelos de AI a través de su *marketplace*.

Fuente: elaboración propia.

Adaptación al estilo de codificación

Los asistentes inteligentes en los IDE pueden **aprender y adaptarse** al estilo de codificación del desarrollador. Esta personalización implica que, a lo largo del tiempo, el asistente haga **sugerencias más alineadas** con la forma en que el programador **escribe y estructura su código.**

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo, Tabnine es un asistente de código que les permite a los desarrolladores entrenar el modelo para ajustarse a su estilo de escritura. De esta forma, las sugerencias y el autocompletado son más naturales y eficientes, ya que aprende de patrones repetitivos y predice con mayor precisión.

Algunos asistentes permiten definir convenciones de estilo, como reglas de sangrado, nombre de variables o uso de comentarios, para que las sugerencias respeten estas preferencias.

Configuración de herramientas de refactorización automática

La refactorización automática es una herramienta poderosa para **mejorar la legibilidad y eficiencia** del código. Los IDE permiten **configurar cómo y cuándo** el asistente debe ofrecer sugerencias de refactorización.

Por ejemplo, en Eclipse, los desarrolladores pueden configurar reglas de refactorización automáticas, como la consolidación de métodos, la extracción de constantes o la eliminación de código duplicado.

© Universidad Internacional de La Rioja (UNIR)

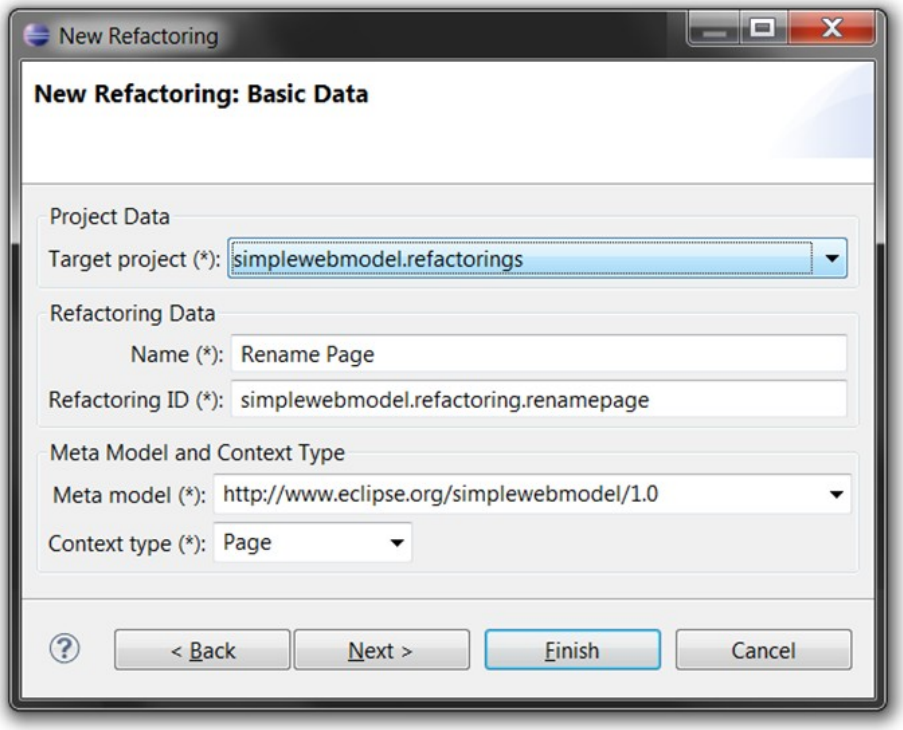


Figura 10. Eclipse permite la creación y configuración de modelos de refactorización. Fuente: Eclipse, s. f.

Integración con herramientas de terceros

A menudo, los asistentes en los IDE se **integran con extensiones** o *plugins* de terceros que añaden funcionalidades adicionales. Estos pueden incluir desde **análisis de seguridad** hasta asistentes específicos para el **uso de tecnologías** como Docker, Kubernetes o bases de datos.

Por ejemplo, VS Code permite integrar herramientas como ESLint o Prettier para analizar y formatear automáticamente el código según las reglas definidas. Estas herramientas se pueden configurar desde el asistente para que funcionen en armonía con las demás funcionalidades del IDE.

© Universidad Internacional de La Rioja (UNIR)

Asistentes contextuales basados en el proyecto

Algunos asistentes son lo suficientemente inteligentes como para **adaptarse al tipo de proyecto** en el que se está trabajando. Esto incluye la capacidad de hacer sugerencias basadas en las **bibliotecas o *frameworks*** específicos utilizados dentro del proyecto.

Por ejemplo, en WebStorm (IDE enfocado en desarrollo web), los asistentes pueden detectar automáticamente si un proyecto usa React, Angular u otro *framework*, y ajustar sus sugerencias de autocompletado y errores basados en ese contexto.

Personalización de asistentes para equipos de desarrollo

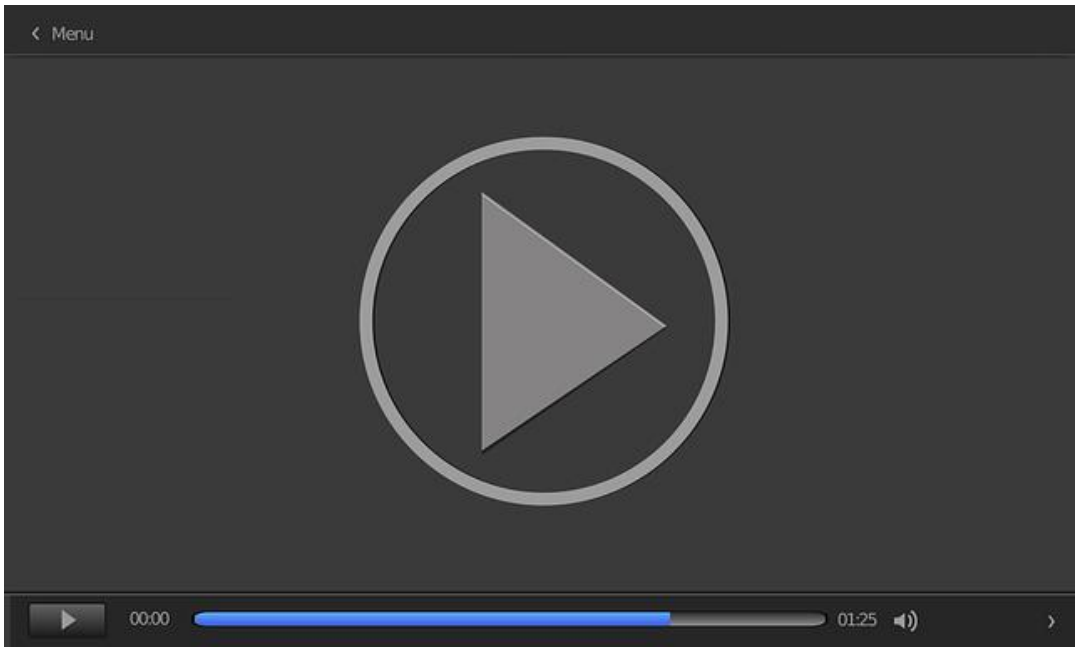
En entornos corporativos o equipos de desarrollo colaborativos, los IDE permiten **compartir configuraciones personalizadas** entre varios desarrolladores. Esto garantiza que todos los miembros del equipo sigan las mismas convenciones y estilos de codificación.

Por ejemplo, en JetBrains TeamCity, se pueden crear configuraciones compartidas que aseguren que todos los desarrolladores utilicen los mismos estándares de autocompletado, de análisis de código y de refactorización.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

En el vídeo *Automatización del proceso Debugging con IA,* se explora cómo las técnicas de IA se aplican en el ámbito del *debugging* automatizado. Se abordarán herramientas y modelos que asisten a los desarrolladores en la identificación y corrección de errores de manera automática. Además, se presentarán casos de uso en donde la IA redujo significativamente el tiempo de depuración, así como los algoritmos que analizan patrones de errores para generar soluciones correctoras.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=dcbd7387-62af-457c-bba4-b21a00998fd6

© Universidad Internacional de La Rioja (UNIR)

4.7. Referencias bibliográficas

Eclipse. (s. f.). *Refactor.* https://wiki.eclipse.org/Refactor

Groenewegen, P. (2020, noviembre 11). *Make your repeated edits faster and more accurate with IntelliCode suggestions.* Microsoft. https://devblogs.microsoft.com/visualstudio/intellicode-suggestion-apply-all/

IntelliJ IDEA. (2024, noviembre 22). *Get results and fix problems.* https://www.jetbrains.com/help/idea/resolving-problems.html

It-Labs. (s. f.). *What's GitHub Copilot and How Devs can Benefit From It.* https://www.it-labs.com/whats-github-copilot-and-how-devs-can-benefit-from-it/

JetBrains. (2025, febrero 3). *Code completion.* Recuperado el 12 de febrero de 2025 de: https://www.jetbrains.com/help/idea/auto-completing-code.html

Kenvel, N. (2023, agosto 30). *How to Use AI-Powered Code Suggestions for Productive Development.* https://www.qodo.ai/blog/ai-powered-code-suggestions-for-productive-development/

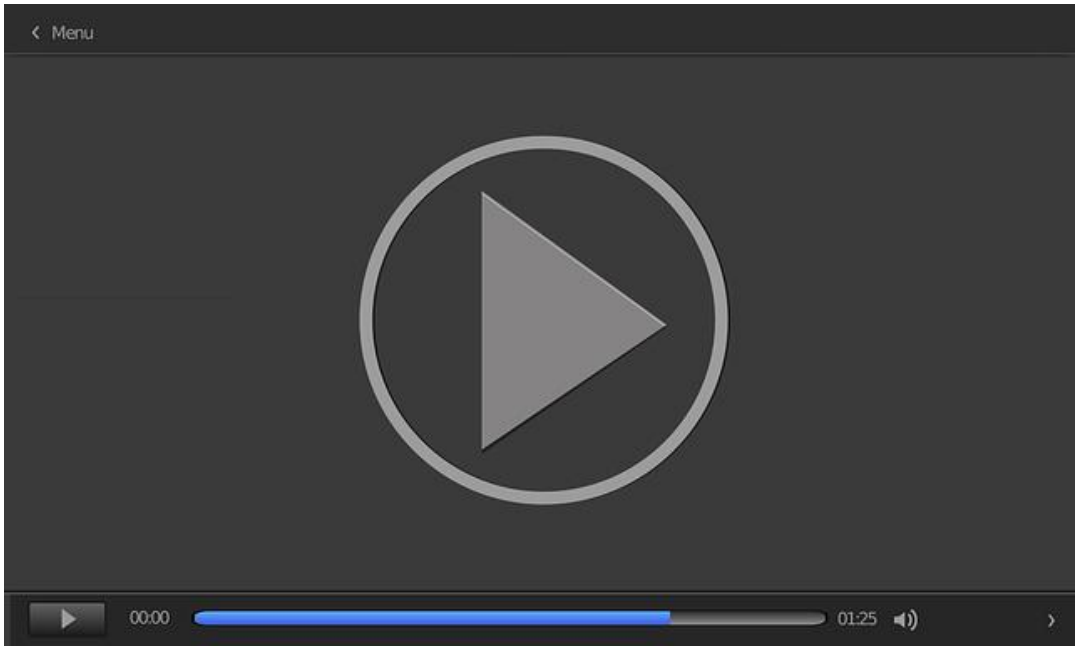
Visual Studio. (2025, febrero 6). *GitHub Copilot in VS Code.* Recuperado el 13 de febrero de 2025 de: https://code.visualstudio.com/docs/copilot/overview

© Universidad Internacional de La Rioja (UNIR)

GitHub Copilot

GitHub. (2022, noviembre 15). *What is GitHub Copilot?* [Vídeo]. YouTube. https://www.youtube.com/watch?v=IqXNhakuwVc

Explicación introductoria sobre GitHub Copilot, una herramienta de asistencia basada en IA que utiliza Codex para ayudar a los desarrolladores a generar código en tiempo real. Se muestran ejemplos prácticos y casos de uso reales.



Accede al vídeo:

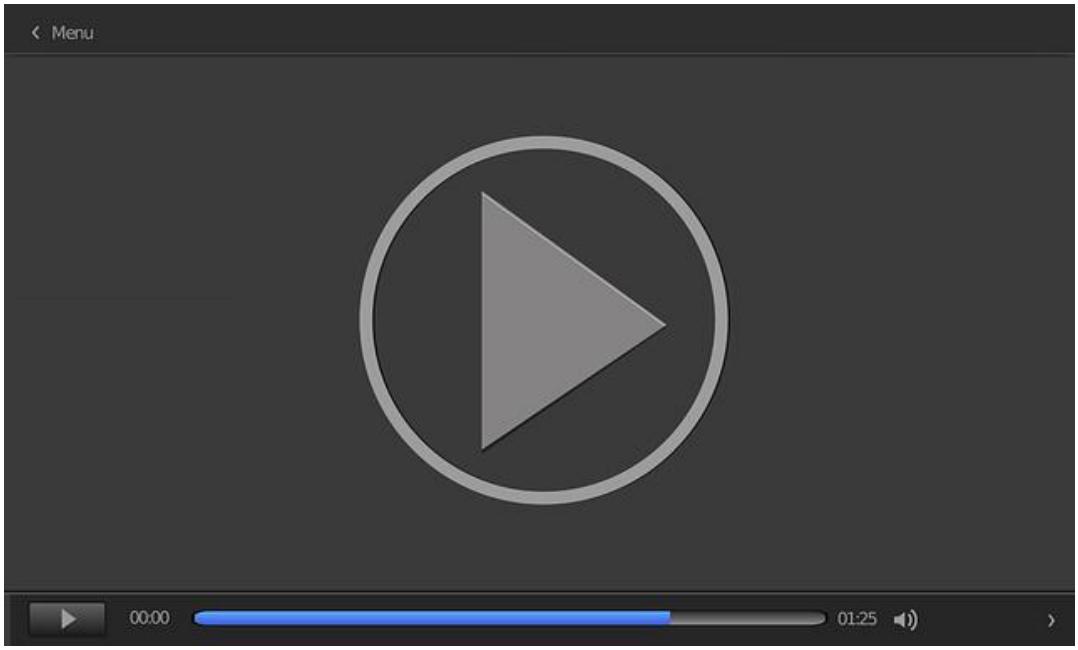
https://www.youtube.com/embed/IqXNhakuwVc?si=qCxUYmOJXxh3yFQe

© Universidad Internacional de La Rioja (UNIR)

JetBrains AI Assistant

JetBrains. (2023, diciembre 7). *JetBrains AI Assistant* [Vídeo]. YouTube. https://www.youtube.com/watch?v=-NnYtfzO7qU

En este vídeo, se explica cómo los asistentes de código en IntelliJ IDEA usan IA para autocompletar fragmentos de código basados en el contexto y las necesidades del desarrollador, para mejorar la productividad.



Accede al vídeo:

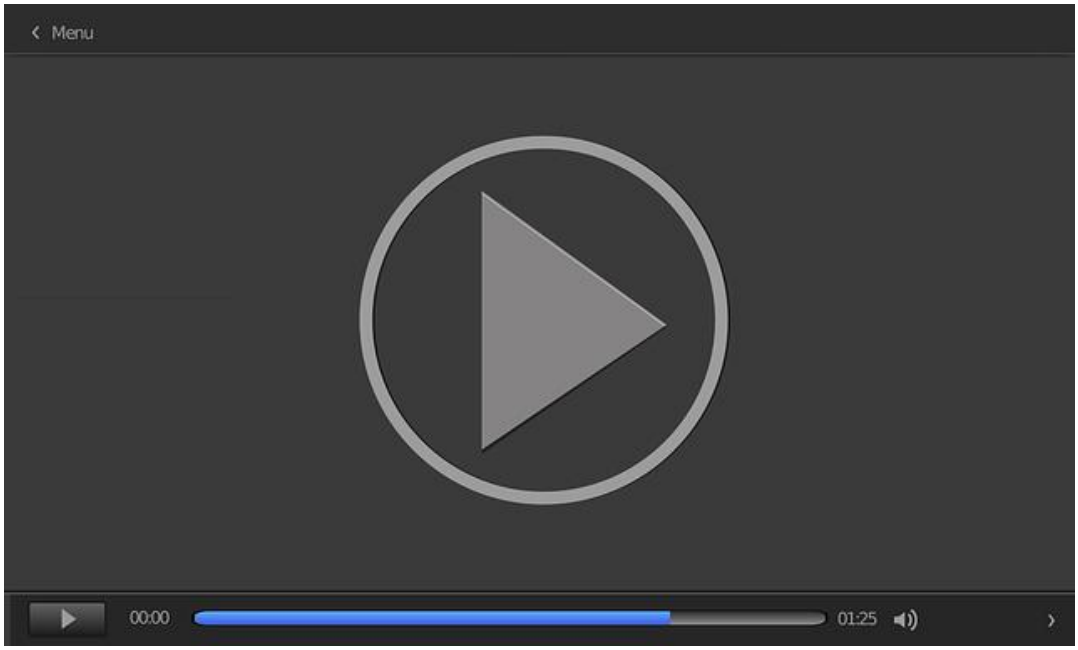
https://www.youtube.com/embed/-NnYtfzO7qU?si=8WRG5BVo6s1BCC2K

© Universidad Internacional de La Rioja (UNIR)

Visual Studio y GitHub Copilot

Microsoft Visual Studio. (2024, marzo 12). *Intro to GitHub Copilot in Visual Studio* [Vídeo]. YouTube. https://www.youtube.com/watch?v=z1ycDvspv8U

En este tutorial, se muestra cómo las herramientas de IA en Visual Studio permiten la detección automática de errores y su corrección en tiempo real, lo que mejora el flujo de trabajo de los desarrolladores.



Accede al vídeo:

https://www.youtube.com/embed/z1ycDvspv8U?si=Hk0KQboV6eTWk6yE

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Qué es GitHub Copilot?

   A. Un sistema operativo para desarrolladores.

   B. Un modelo de lenguaje natural.

   C. Un asistente de código basado en IA que sugiere código en tiempo real.

   D. Un editor de texto avanzado.

2. ¿Qué es una de las principales funcionalidades de un IDE con IA?

   A. Ejecutar videojuegos.

   B. Realizar pruebas automáticas de *software.*

   C. Sugerir y completar código con base en el contexto.

   D. Desplegar proyectos en la nube automáticamente.

3. ¿Cuál es una de las ventajas de la detección automática de errores en un IDE?

   A. Facilita la creación de interfaces gráficas.

   B. Aumenta el tamaño del código.

   C. Reduce el tiempo de depuración y mejora la calidad del código.

   D. Convierte el código a otros lenguajes automáticamente.

4. ¿Qué modelo de IA utiliza GitHub Copilot para generar código?

   A. GPT-3.

   B. BERT.

   C. LSTM.

   D. CNN.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Qué tipo de errores pueden detectar los IDE avanzados con IA?

A. Solo errores de sintaxis.

B. Errores de sintaxis, de semántica y de lógica.

C. Solo errores lógicos.

D. Solo errores de formato.

6. ¿Qué es el autocompletado basado en contexto en un IDE?

A. La predicción de errores en tiempo real.

B. La sugerencia de fragmentos de código basados en la línea de código actual.

C. La detección de variables duplicadas.

D. La traducción automática de código.

7. ¿Cuál de los siguientes es un ejemplo de personalización de un asistente en un IDE?

A. Cambiar el color del tema.

B. Configurar las sugerencias de autocompletado para priorizar ciertas librerías.

C. Crear una nueva clase automáticamente.

D. Desinstalar el IDE.

8. ¿Qué característica permite que un IDE, como IntelliJ IDEA, sugiera mejoras de código?

A. El análisis estático de código.

B. La utilización de diagramas UML (lenguaje unificado de modelado).

C. La detección de vulnerabilidades.

D. La generación automática de gráficos.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Qué modelo es responsable de la capacidad de traducción entre lenguajes en algunos IDE con IA?

A. Codex.

B. LSTM.

C. Perceptrón.

D. SVM.

10. ¿Qué permite hacer el autocompletado en tiempo real de los IDEs basados en IA?

A. Crear gráficos automáticamente.

B. Proporcionar fragmentos de código basados en el contexto de lo que se está escribiendo.

C. Traducir documentos.

D. Analizar datos en hojas de cálculo.

11. ¿Qué ventaja tiene la personalización de asistentes de código?

A. Reducir el uso de memoria en el IDE.

B. Facilitar el flujo de trabajo y adaptarse al estilo del desarrollador.

C. Aumentar el tamaño de las sugerencias.

D. Eliminar errores de compilación automáticamente.

12. ¿Qué funcionalidad ofrece GitHub Copilot en relación con la documentación del código?

A. Detecta fallos en la estructura de los documentos.

B. Sugiere comentarios y documentación basada en el código escrito.

C. Genera hojas de estilo CSS.

D. Convierte el código en texto plano.

© Universidad Internacional de La Rioja (UNIR)

13. ¿Cuál es el beneficio de usar la IA para la corrección automática de código?

A. Mejora el aspecto visual del código.

B. Permite detectar errores y corregirlos en tiempo real.

C. Aumenta la longitud del código generado.

D. Facilita la ejecución de proyectos en servidores remotos.

14. ¿Qué IDE es conocido por su integración avanzada con GitHub Copilot?

A. Eclipse.

B. IntelliJ IDEA.

C. VS Code.

D. NetBeans.

15. ¿Qué técnica utilizan los IDE con IA para analizar el flujo de un programa y sugerir cambios?

A. *Machine learning* supervisado.

B. Análisis dinámico de código.

C. Redes neuronales convolucionales.

D. Transformación de datos.

16. ¿Qué ventaja ofrece la integración de control de versiones en los IDE?

A. Gestionar gráficos automáticamente.

B. Realizar commits y fusiones sin salir del IDE.

C. Generar bases de datos.

D. Implementar algoritmos de *machine learning.*

© Universidad Internacional de La Rioja (UNIR)

17. ¿Qué capacidad les permite a los IDE detectar vulnerabilidades de seguridad en tiempo real?

A. El análisis estático y el *machine learning.*

B. La generación de interfaces gráficas.

C. La creación automática de comentarios.

D. La optimización de algoritmos.

18. ¿Qué tipo de refactorización pueden sugerir los asistentes de código?

A. Cambiar la fuente del editor de código.

B. Renombrar variables y optimizar importaciones.

C. Eliminar las pruebas unitarias.

D. Añadir nuevas clases automáticamente.

19. ¿Qué tecnología impulsa el asistente de código de GitHub Copilot?

A. GPT-3.

B. VGGNet.

C. YOLO.

D. CNN.

20. ¿Qué permite el uso de plantillas de código en un IDE?

A. Crear bases de datos en tiempo real.

B. Insertar fragmentos de código predefinidos para acelerar el desarrollo.

C. Generar gráficos automáticamente.

D. Ejecutar pruebas unitarias.

© Universidad Internacional de La Rioja (UNIR)